

How to Compare the Security of Code Written by Humans to LLM-generated Code

Jasmine Egli¹, Rebecca Balebako^{1,*}

¹Zurich University of Applied Sciences

Abstract

Large language models (LLMs) are rapidly transforming how software is created and maintained. Comparing LLM-generated code against human-written standards is essential to determine whether these new tools uphold or erode the security baselines established by professional developers. Yet, we lack a standardized method for empirically comparing the security of code produced through human-LLM collaboration against LLM-only, or traditional human-only methods. To facilitate this, we propose an automated framework for conducting comparative studies across human-only, LLM-only, and hybrid conditions. Our approach automates the logging of prompts, timing, and experimental settings, measuring outcomes through multi-dimensional static and dynamic quality analysis. We provide an open-source implementation of this framework to ensure that future researchers can conduct reproducible, species-fair experiments. Importantly, we validate the framework via a feasibility study, providing an experimental blueprint for “species-fair” comparisons between human and AI subjects. By sharing lessons learned, we establish a foundation for empirical research on human and LLM-generated code for software security.

Keywords

Security, Secure Code, LLMs, Quality of LLM-generated Software Artifacts

1. Introduction

Large language models (LLMs) have fundamentally altered software development. While LLM tools offer significant efficiency gains, security remains a primary, unresolved concern [1]. Current literature offers disparate conclusions on the safety of LLM-generated code, often because outcomes are artifacts of inconsistent evaluation methods and contexts.

There is relatively sparse literature on how humans and LLMs compare in producing secure code. To limit confounding factors, comparing human and LLM-written code requires a “species-fair” approach [2]. Empirical research that attempts to compare the security of LLM versus human code should follow best practices in Developer Centered Security research and best practices in LLM evaluation, while striking the balance in what is a fair comparison.

Furthermore, evaluations of human and LLM-written code are often confounded by non-deterministic outputs, varying task constraints, and a reliance on subjective or high-friction manual reviews. Without a reproducible and multi-dimensional measurement pipeline, it is difficult to determine whether observed security flaws are merely artifacts of inconsistent experimental design. To address these inconsistencies, future empirical work must employ precise, species-fair designs that account for these influences. There is an urgent need for an automated, deterministic framework that can bridge this gap and enable researchers to isolate the variables that influence code security across human, LLM, and hybrid generation paradigms

Our developed framework proposes a path to bridging the gap. It offers researchers a method to compare the functionality and security of human-written code and LLM-generated code. This framework can help designers, security researchers, and tool developers identify conditions and variables that influence code security across different code generation paradigms. By providing humans and LLMs with identical exercises and executing the results in identical environments, and then utilizing code

Joint Proceedings of the STAF 2026 Workshop, LLM4SE Rennes, France, June 29- July 3, 2026

*Corresponding author.

ORCID: 0009-0002-0160-5709 (J. Egli); 0000-0001-9862-0790 (R. Balebako)



© 2026 Copyright @ 2026 for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0)

quality metrics to understand code security, the framework allows for a direct assessment of how human logic compares to the capabilities of modern models.

We evaluate the framework on curated human code and LLM-generated code to demonstrate the feasibility of this approach. We provide lessons learned for experimental design, task creation, and security metrics to ensure reproducibility. Our contribution lies in the identification of experiment design issues, providing a roadmap for future researchers to align LLM secure programming capabilities with commensurate human skill levels in a way that is scientifically defensible.

1. **Instructional Symmetry:** Identifying the prompt engineering necessary to maintain a fair comparison between human and LLM instructions.
2. **Model Reliability:** Documenting how specific model behaviors can break automated evaluation pipelines.
3. **Analysis of Experimental Attrition:** Categorizing the "leaky pipeline" where samples are lost to malformed outputs or execution failures, providing a blueprint for the sample over-provisioning required for statistical validity.

The next sections are as follows. In the **Related Work** section, we give an overview of relevant work in evaluating secure code. In the **Framework Implementation** section, we describe the design decisions made in designing the framework tool and outcomes. In the **Feasibility Study** we describe the feasibility design and lessons learned. Finally, in the section **Discussion**, we cover limitations, future work, and ethics.

2. Related Work

2.1. Measuring the security of code

Despite its complexity, quantifying security is essential for comparative code evaluation. The focus of this work is on source code, as opposed to broader systems of security, as this is an area LLMs are currently helpful.

Secure code evaluation can be described in three rough groups: 1. Researchers manually review and rate code [3] [4]. This is not only labor-intensive but requires checks for reliability. 2. LLMs act as judges to evaluate the security of code, which may introduce hallucinations or other errors. 3. Use tools such as static analysis to evaluate code for specific metrics, which can introduce the bias of any specific tool into the evaluation [5]. To focus on deterministic, repeatable outputs, we use focus on the third option.

2.1.1. Run-time Analysis for Errors and Correctness

Dai et al. [5] examine LLM-generated code, and show the need to measure both functionality and security of code. Correctness include error rates and functionality. Code should run without failure and it should complete the required task.

Frequent runtime errors can indicate fragile code. High error rates during execution suggest a lack of robust input validation, which is the primary entry point for injection attacks. [6] demonstrated that traditional software fault metrics are strongly correlated with security vulnerabilities, suggesting that "buggy" code is statistically more likely to be "vulnerable" code.

Fuzzers are an example of run-time analysis [7]. By running the program with various inputs, they measure whether the programs fail or produce warnings. This can help find bugs and vulnerabilities in specific contexts. Fuzzing is a reasonable extension of this work.

2.1.2. Static Analysis for Code Quality

Static analysis automates code evaluation against a predefined rule set, enabling scalable, fast, repeatable quality checks. However, static analysis cannot include contextual nuances, runtime behavior, and

emergent properties. Despite these limits, static analysis integrates into automated pipelines and has relevance to code security [8, 9, 10]. Key metrics include *Cyclomatic Complexity*: (McCabe), which measures the number of linearly independent paths through a program’s source code and acts as predictor of security vulnerabilities [6]. *Mean function length*: or lines of code per function is another indirect measure of security. Long functions tend to “leak” state, where variables intended for one task are accidentally reused for another, leading to memory corruption or sensitive data exposure [11, 12, 13].

2.1.3. Manual Analysis

Manual analysis is the process where human experts meticulously inspect source code to identify security flaws, logic errors, and violations of secure coding standards. Unlike automated tools that rely on predefined patterns, manual analysis leverages human understanding of the application’s business logic to uncover “context-aware” vulnerabilities [14]. Manual analysis has pros and cons, with some research finding the effectiveness to be highly variable [15]. While not a part of our framework, it would be possible to additionally include manual evaluation scores in the final assessment.

2.1.4. Developer Experience

Some studies on code security focus on human experience, such as time to completion [16] or self-reported confidence in the code [17]. Longitudinal work has also included total time to develop and review the LLM-assisted code generation [18]. Our framework does not include developer experience metrics, as we focus on LLM-generation and automated measurement.

2.2. Developer Centered Security

Code security is shaped by environment in which the code is produced. Developer-Centered Security (DCS) is a subset of the usable security community. This field includes over a decade of work to understand what conditions influence secure code, and offer insights into the contexts in which developers produce secure (or insecure) code [3] [4].

DCS studies generally use one of three experimental methods. Qualitative research can include interviews or small surveys (including [19, 20, 21, 22]). Observational research involves examining existing repositories [23, 24]. Experimental studies ask developers to write code with specific functional requirements under specific conditions. In some cases, developers are asked to come to the lab, and in others they can perform the task remotely (for example [25]). Recruiting software developers is particularly challenging for software engineering research [26, 27], and usable security is no exception [28, 29]. Because developer recruitment is a significant bottleneck, our work provides the necessary foundation to identify and resolve experimental flaws before engaging human subjects.

Recruiting software developers is particularly challenging for software engineering research [26, 27], and usable security is no exception [28, 29]. Participants for studies can be drawn from freelance platforms, crowdsourcing, recruitment in developer communities, or directly within cooperative companies [30]. As recruitment is already challenging, our framework strives to ensure that code output from each developer measured in a high-quality, reproducible manner.

Current empirical evaluations of LLM-assisted coding in DCS have yielded fragmented results, with findings on security and productivity varying significantly across different study designs. Sandoval et al. [31] used a randomized control trial to understand how novice programmers were impacted by LLMs. Overall, they found LLMs’ assistants had small “but consistent” benefits to functionality of the code without adversely impacting the security of the code. Perry et al. [17] discovered that participants with access to an LLM assistant wrote less secure code than those without access to an assistant. Belozarov et al. [32] explored the capabilities of ChatGPT in detecting security issues but concluded that LLM-generated code tends to have more vulnerabilities than human-written code. Becker et al. [18] conducted a randomized controlled trial to understand how LLM tools affect the productivity of experienced open-source developers and found that, contrary to expectations, the completion time of development tasks actually increased by 19%.

2.3. Comparing Human to LLM code

Direct comparisons of code by humans and by LLMs are often confounded by asymmetrical testing conditions, necessitating a more balanced approach to evaluating human and LLM-generated software. Firestone [2] advocates for “species-fair” evaluations, emphasizing the necessity of equivalent constraints when benchmarking human and machine performance. Drawing parallels from computer vision, the author argues that direct assessments are only valid when both agents operate under symmetrical environmental and task-based parameters.

Prompt engineering is the practice of refining the input queries of “prompts” to LLMs to meet certain goals [33]. Bruni et al. [34] suggests the prompts can have significant impact in the security of LLM-generated code, as measured by static analysis for specific vulnerabilities. State-of-the-art methods for secure-code generation from LLMs describe fine-tuning models with secure and insecure code [35, 36]. A comparable training method for humans may include a comprehensive training or a course on secure code. Other secure code generation methods include reasoning loops in which the model is prompted to re-think their code with security in mind, perhaps based on results from a static analyser [37]. Similar tasks for humans would likely include multiple steps in which coders are specifically asked to review their code with security in mind.

Previous research on comparing humans and LLMs have not relied on species-fair comparisons. Molison et al. [38] integrated Python solutions from open source repositories to test various LLM configurations to measure maintainability, reliability, and remediation effort. However, the LLM prompts included zero-shot, few-shot, and fine-tuning. Arguably this would be species-fair if humans also had multiple attempts to write the code. Cotroneo et al. [39] analyzed over 500,000 code samples in Python and Java to compare human developers to LLMs, also using static analysis for code security and measuring code complexity. The LLMs were prompted to generate code by giving them the docstring and function signature that was extracted from the human code. This arguably means the LLMs started with a different task than the humans. Licorish et al. [40] used the same prompt for humans and LLMs, therefore being more “species-fair.” With a dataset of 72 software engineering tasks, they used static analysis tools to measure coding standards, complexity, and security vulnerabilities. However, only one human was responsible for the full human-generated code dataset, which introduces concerns about representativeness of human coders. The findings across these papers are contradictory in regards to whether LLMs or humans are “better” or “more secure”, and some were task dependent.

A further issue emerges in LLM to human code comparison. Researchers analyzing human and LLM generated code must contend with noise in the experimental data. For example, both LLMs and humans are inherently non-deterministic, meaning identical tasks and setups can yield various code results, even from the same LLM or human [41, 42, 43]. Work systemizing such developer centered security study research has provided information to estimate the statistical power of developer surveys [44]. Research comparing humans to LLMs will need to determine also the power of any LLM-generated code samples, due to residual nondeterminism in LLM internals [45] [46] [47].

3. Framework Implementation

3.1. Creating and Collecting Code

Comparing human-created code to LLM-generated code requires acquiring code from humans and prompting LLMs to generate code. Researchers may ask participants and LLMs to write code for different tasks. We refer to these human-language instructions as “exercises.” For example, an simple exercise might be to “Write python to generate the first 10 values in Fibonacci sequence of numbers.” For LLMs, the exercise is often called a “prompt.” Our framework is designed to track and compare results for multiple exercise definitions, and assume the same exercise is given to both humans and LLMs. After a human or LLM receives the exercise, they should generate executable code.

3.2. Framework Design

The architecture overview of the implementation is displayed in Figure 1. In general, this framework resembles many of the related work examples, with additional emphasis in reproducibility and species-fair design. The framework serves a dual purpose: first, as a standardized interface for code generation across various LLMs, and second, as a deterministic execution and evaluation pipeline for security measurement. Key properties of a deterministic system include: (1) clear definitions of input and output variables, (2) precisely known initial conditions, (3) cause-effect relationships, (4) predictability, (5) continuity, (6) reproducibility, (7) modeling, and (8) time independence. This architecture ensures that our measurements of code quality remain reproducible to the extent possible.

The framework is implemented in Python and available on Github [48]. All artifacts during the execution of the pipeline are persisted in a centralized SQLite database.

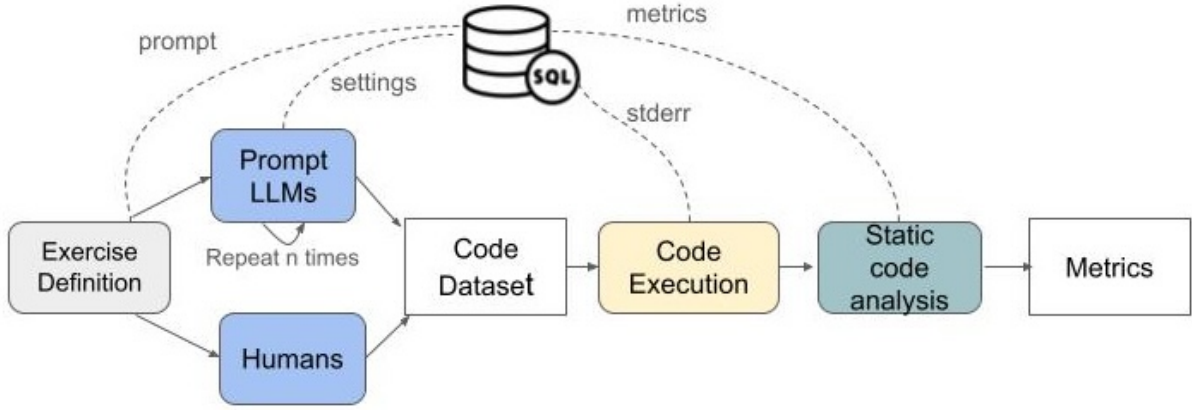


Figure 1: Framework design: Exercise definition as prompt for LLMs and humans, the output is executed in a container, and static analysis is run. All settings and environment variables are saved the database.

Exercise definition: The programming exercises (tasks or prompts describing what code to write) are stored as version text-files in a git repository and then imported into the prompt table in the database. This allows clear exercise inputs over runs that are identical and documented.

Code generation: Automated model invocation with strictly defined input-output parameters ensures a traceable audit trail for all generated code artifacts. For LLMs, we used OpenAI API with fixed request parameters (client, model, temperature, top p, max tokens, frequency penalty and presence penalty) and defined response parameters (timestamp, output text, response id, request id, finish reason, latency ms, output tokens, total tokens). The parameters were fixed across models and defined in A. Generated code is captured in a defined output format to minimize variability.

This step is repeated for a fixed number of times, creating a number of code outputs for each exercise and model, before moving to the next step in the pipeline.

Code execution: To ensure consistent initial conditions, both human and LLM-generated code are executed in isolated Podman containers with a fixed set of local dependencies. We further mitigate environmental non-determinism by restricting tasks to self-contained logic that does not rely on external network services.

Code evaluation: As described in Table 1, we used a collection of metrics to measure correctness, clarity, and security of the code output. A rule-based assessment was performed with static Ruff linter [49].

3.2.1. Reproducibility and Auditing

The pipeline captures metadata to create traceable artifacts linking specific prompt inputs to quality outputs. Metadata includes: model parameters, timestamps, execution logs, and security results. This

Table 1
Quality Metrics and Criteria

Quality Criteria	Criteria	Metric	Description
Correctness		Pass (1) / fail (0)	Output of the code execution is evaluated automatically to determine if the output is correct or not.
Clarity		Cyclomatic complexity	Measures independent paths through the code sample. High complexity indicates hard-to-understand logic.
		Function length	Counts the number of lines in a function. Large functions are harder to understand.

structured environment ensures the reproducibility of deterministic properties, such as initial conditions and cause-effect relationships.

However, while the framework facilitates characterization through repeated trials, absolute predictability remains hindered by the intrinsic non-determinism of LLM internals [47] and external service variability. To preserve the integrity of these trials, execution and evaluation results remain inaccessible to the models, preventing the LLMs from learning across multiple iterations and ensuring that each generation is independent.

3.2.2. Containers for Security and Reproducibility

We use Podman (an open-source, daemonless container engine) for reproducibility [50], but they serve a secondary purpose in protecting the researchers’ environment. Evaluating code security requires executing potentially malicious or malformed logic. We avoid these pitfalls by routing all generated code through a rootless Podman containerization layer.

3.3. Framework Results

The framework implements four primary controls to isolate variables identified in Developer-Centered Security (DCS) literature while adhering to the species-fair doctrine [2].

3.3.1. Species Fair Integrity

Environmental Symmetry: To ensure fair constraints, we utilize Podman-based containerization. This guarantees that both human and LLM-generated code are evaluated within identical operating systems, library sets, and hardware allocations.

Instruction Parity: Functional requirements and security constraints must be mirrored exactly between human instructions and LLM system prompts. This prevents prompt-tuning bias, where one agent receives more architectural guidance than the other.

Contextual Calibration: DCS research highlights participant expertise as a critical determinant of security. The framework allows researchers to map model capabilities (e.g., base vs. reasoning models) to commensurate human experience levels (e.g., students vs. seniors).

Functional Quality Gates: To avoid the common pitfall of evaluating the security of non-functional code, the pipeline enforces sequential evaluation. Code must pass functional unit tests before undergoing security analysis, ensuring results are not confounded by syntax or logical failures.

Isolation and Traceability: Each iteration generates a unique audit trail linking prompts to outcomes. The environment resets for every trial to prevent data leakage or cross-iteration learning, ensuring independent results.

3.3.2. Research Utility and Framework Benefits

The framework provides a flexible, extensible foundation for comparative studies across diverse code-generation paradigms.

Multi-dimensional Assessment: The pipeline automates the evaluation of execution, code clarity, and security. We leverage the Ruff linter (800+ rules), allowing researchers to toggle security patterns relevant to their specific exercises (e.g., omitting SQL injection checks for mathematical tasks).

Reproducibility : By minimizing measurement variance in inherently non-deterministic LLM environments, the framework provides a stable baseline.

Operational Efficiency: Designed for accessibility, the toolchain utilizes ubiquitous technologies (Python, SQLite, Podman). During our feasibility study, OpenAI API costs were minimal (approx. \$10), and the containerized architecture protected the host machine from potentially malicious code execution.

Privacy: To protect human participants , the framework is a “closed loop” It does not feed code back into LLMs for training or prompts, preventing the proliferation of private information (such as humans including their identifiers in the code).

4. Feasibility Study

To demonstrate the framework’s application in assessing both LLM-generated and human-authored code, we executed a feasibility study focused on Python security and algorithmic challenges.

4.1. Exercise selection

Thirteen coding exercises were chosen as prompts. These prompts cover Python security exercises and Advent of Code (AoC) 2024 challenges.[51]. These tasks span difficulty levels from beginner to expert and cover diverse problem types (algorithmic puzzles, parsing, data-structure task) as well as security-focused cases. These challenges require both syntactic correctness and algorithmic reasoning, expose security and robustness issues, and allow deterministic validation of correctness via predefined input/expected output pairs. Where available, property-based unit tests were used to verify and stabilize basic building blocks of the code base [52].

4.1.1. Human Code Data sources

A set of human reference solutions was included in execution and quality assessment steps (see Figure 1). For w3resource exercises, only one online solution was available. For Advent of Code 2024, hundreds of solutions exist on Github. Approximately five high-ranking Python solutions were selected from the public leader board and extracted from their repositories.

4.1.2. LLM Code Generation

Exercises were copied from the challenge websites and used as prompts. For this process input data are embedded in the prompt text file, as the LLMs cannot load external directories.

Five LLM model variants were compared: gpt-4.1, gpt-4o-mini, gpt-5.1, gpt-5-mini, and gpt-5-nano. According to the OpenAI documentation, these models differ in coding reasoning, determinism, latency, cost per run, error profile or failure modes. [53]

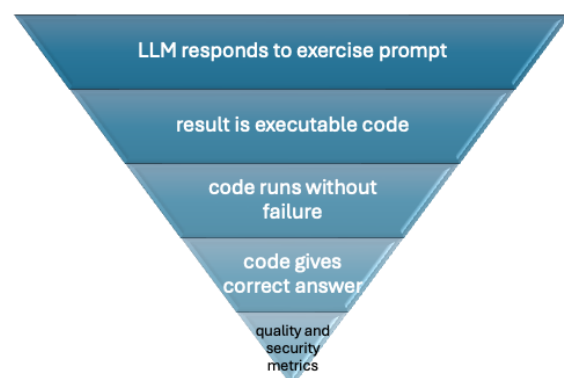


Figure 2: Cascading errors: Results at each step may be disqualified from inclusion in the next analysis.

Each exercise and model combination can be executed in the automated pipeline, as described in 1, multiple times. Prompt ingestion, parametrization (temperature, top p, max tokens, frequency penalty and presence penalty), and input token number were stable across runs. All requests to generate code completed without error.

4.1.3. Handling Errors

In this analysis, code is analyzed in multiple steps, as shown in Figure 2. For example, the LLMs did not always output executable code; sometimes they just returned the answer or human text intermingled with code snippets. We did not do further execution or testing on non-code results. In these cases, the sample was removed from further testing. Some of the executable code failed to run, or ran and produced incorrect answers. If it failed, the analysis stopped for that sample. The assessment for cyclomatic complexity, function length and security metrics was done for correct code results only.

4.2. Feasibility Study Results

We utilize a pilot sample to examine the nuances of species-fair task constraints and automated security evaluation. Rather than providing definitive security conclusions, these preliminary results stress-test the evaluation pipeline, ensuring the framework can reliably support the complex, species-fair research it was built to facilitate.

4.2.1. Data Analysis

Each exercise was generated 60 times per model to minimize within-task variance and approximate independent draws [54]. 60 repetitions were intended to strike a balance between capturing variability in model output and practical resource and time constraints. However, as described previously, we were not able to run all samples. In our case, the total test should have 3,900 data points (5 models * 13 exercises * 60 times), but we were only able to execute 3,832 samples (98%) of LLM-generated code, and 2,615 produced correct output (67%). Furthermore, some models never produced correct code for some exercises. This leads to a skewed dataset which violates many common distribution assumptions. Assumptions for linear regression, ANOVA, and poisson regression did not hold due to the dispersion and distribution of these errors.

The selection of top-rated human code samples for this feasibility study introduced an inherent ceiling effect, as all human-authored entries executed without functional errors. These samples are not intended to represent a statistically random distribution of developer performance. Rather, the impact of this selection process reinforces our primary thesis: that future comparative research requires a standardized framework built from the ground up to account for and mitigate such experimental biases.

4.2.2. Correctness

For each exercise, the “correct” answer was a pre-defined string. We verified whether the executed code generated this correct answer. Correctness varied greatly across the exercises. Some exercises were solved consistently across all models, others showed more model-dependent performance. Correctness was near-100% for gpt-4.1, strong but task-sensitive for gpt-4o-mini, and more variable for the gpt-5 family (with gpt-5.1 out-performing gpt-5-mini/nano).

To account for the bounded nature and unequal variance of the correctness scores (0 to 60), we employed a Binomial Generalized Linear Model (GLM) using a logit link function to model the correctness. The relative influence of prompt id and model was then quantified by comparing the increase in model deviance upon the removal of each variable, identifying the factor that contributed most significantly to the model’s overall fit. We find the exercise itself was more predictive of correct output than the different models.

Predictor	Δ Deviance	df	p-value	Influence Rank
Prompt ID	1693.59	12	< 0.001	1st
Model	397.81	4	< 0.001	2nd

Table 2

Analysis of Deviance for Correctness: The higher variance for the prompt id indicates that the exercise had more influence on correctness than the model.

4.2.3. Failure Mode

Analysis of failure modes revealed two dominant error types: input/output and structural errors. (1) “No output” error type meaning that the executed code did not print a correct standard output. Analysis of the code showed that this error is caused by its improper input handling. The generated solutions assumed external files (e.g., password.txt), expected manual input, omitted input assignment, or printed natural-language answers instead of executable code. (2) Algorithmic and structure errors: The LLM generated functioning code but the output was wrong (“WrongLogic”), or the script was aborted due to “IndexError”, “RecursionError” or “NameErrors”.

Aggregated error counts of incorrectly solved challenges are shown in 3.

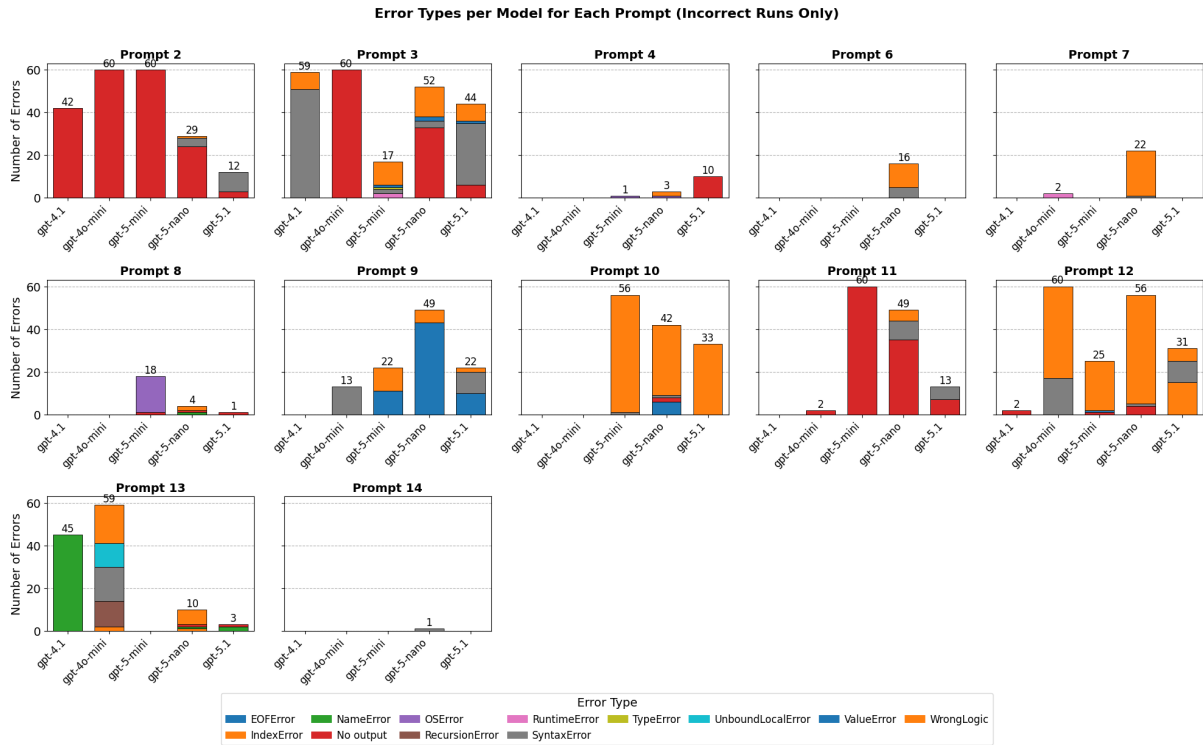


Figure 3: Failure Modes: The types of errors across models and runs. WrongLogic or No Output were the most common errors.

4.2.4. Clarity

Code clarity is characterized by cyclomatic complexity and function length. Comparative analysis of correctly solved runs highlighted distinct stylistic differences between models and human authors. See Table 3. Human-authored references generally exhibited lower McCabe cyclomatic complexity than the models, despite the functions being longer. The GPT-4 series produced less complex solutions than the GPT-5 series on harder tasks. Similar to McCabe cyclomatic complexity, we assessed the quality metric “function length”. Typical mean values varied from 4 up to 20 lines, with CLT standard errors ranging from ± 0.3 to ± 2.0 depending on sample size (n).

model	gpt-4.1	gpt-4o-mini	gpt-5-mini	gpt-5-nano	gpt-5.1	human
McCabe	4.6 ± 0.1	3.3 ± 0.0	5.2 ± 0.1	4.2 ± 0.1	4.5 ± 0.1	2.9 ± 0.2
Function Length	9.2 ± 0.3	9.3 ± 0.1	7.4 ± 0.7	13.5 ± 0.4	6.4 ± 0.1	10.7 ± 0.2

Table 3

Complexity Scores: Mean value of cyclomatic complexity (McCabe) with CLT standard error for correctly solved challenges only across different models and human generated references. Mean value of function length with CLT standard error for correctly solved challenges across different models and human generated references.

4.2.5. Security

The total number of security issues reported was small, limiting the statistical conclusion. However, the data suggests that security error types were clustered by exercise rather than the model family, and similar errors occurred within an exercise across models. While the counts were small for the human-authored code, it appears that errors that the models commonly make in an exercise were also seen in the human-generated code.

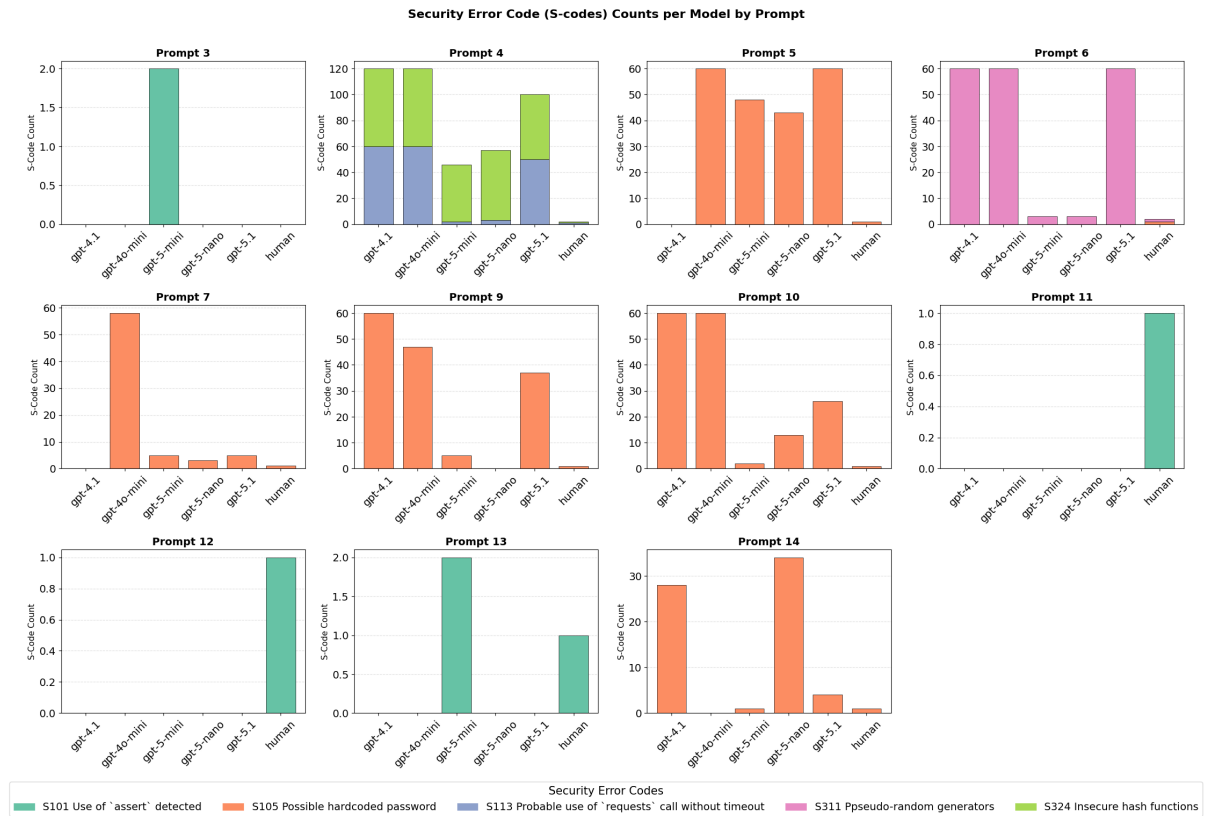


Figure 4: Security Errors: The types of security errors flagged by Ruff per model and prompt.

Figure 4 summarizes security rule violations across models and human-generated code. No issues were flagged for prompt 2 or 8.

5. Discussion

There is still work to do to understand where humans versus LLMs should be writing code for security. This work does not claim to offer a final answer that one or the other is “better.” Instead, we offer a framework so that more work can be done. The framework is designed to eliminate confounding

factors, ensuring that the results reflect true security differences rather than artifacts of a mismatched experimental design.

This framework prioritizes a species-fair way to compare code written by humans versus LLM by giving LLMs and humans the same prompts and evaluating them on the same metrics. Such a prompting style may seem suboptimal for either the humans or the LLMs, but allows for comparable results. In this manner, we can compare apples to apples to better understand strengths and weaknesses of secure code output for similar tasks.

The feasibility study offers lessons learned for future experiments. The feasibility study can be used to help researchers estimate statistical power or anticipate experimental concerns before comput-time is expended or human subjects are recruited.

5.1. Limitations

The following limitations in the framework were observed:

- Task coverage and metric sensitivity: some quality metrics, notably security, did not reach statistical significance for the selected problems. The framework identified that current sample sizes in the literature are likely underpowered.
- While the current reference implementation is Python-specific, the containerized architecture and evaluation logic are language-agnostic.
- This work was carried out in late 2025. The rapid increase in model capability may influence future work.

5.2. Future Work

Future research should transition toward task-specific, focused experiments designed to identify the exact points in a workflow where LLMs provide the most value and where human intervention remains essential.

Exercise Selection: Our results suggest that exercise selection influences outcomes more than model choice, necessitating a focused understanding of how specific tasks elicit vulnerabilities.

Prompt Engineering: Prompt engineering is crucial in providing a fair comparison. For example, given the exercises instructions and context, humans may understand to write code, but LLMs may need this explicitly stated. However, researchers must maintain a balance between prompt optimization for LLMs and human readability.

Model Selection: The quality of models overall set a baseline performance. However, increased model reasoning capacity did not inherently translate to higher-quality code. There are opportunities for future work here.

Metric Selection: The framework supports a modular selection of metrics tailored to specific research questions. We report results separately to ensure they remain adaptable to diverse research goals. Developing a single score would require a subjective weighting system that may not align with every study.

5.3. Ethical Considerations

Common ethical concerns of this study were discussed by the researchers and considered to be acceptable. No vulnerabilities in production code or in real systems were discovered in this work. There are no direct military applications. No human subjects were used in this work. The data collection adhered to the terms of service of the respective platforms and follows established norms for the use of publicly available datasets in software engineering research [55].

Declaration on Generative AI

During the preparation of this work, the author(s) used Gemini in order to: Paraphrase and reword, improve writing style, draft code, and simulate peer review.

References

- [1] L. C. Ramírez, X. Limón, A. J. Sánchez-García, J. C. Pérez-Arriaga, State of the Art of the Security of Code Generated by LLMs: A Systematic Literature Review, in: 2024 12th International Conference in Software Engineering Research and Innovation (CONISOFT 2024), 2024, pp. 331–339. doi:10.1109/CONISOFT63288.2024.00050.
- [2] C. Firestone, Performance vs. competence in human–machine comparisons, *Proceedings of the National Academy of Sciences* 117 (2020) 26562–26571. doi:10.1073/pnas.1905334117.
- [3] M. Tahaei, K. Vaniea, A Survey on Developer-Centred Security, in: 2019 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW), 2019, pp. 129–138. URL: <https://ieeexplore.ieee.org/document/8802434>. doi:10.1109/EuroSPW.2019.00021.
- [4] Y. Acar, S. Fahl, M. L. Mazurek, You are Not Your Developer, Either: A Research Agenda for Usable Security and Privacy Research Beyond End Users, in: 2016 IEEE Cybersecurity Development (SecDev), IEEE, Boston, MA, USA, 2016, pp. 3–8. URL: <http://ieeexplore.ieee.org/document/7839782/>. doi:10.1109/SecDev.2016.013.
- [5] S.-C. Dai, J. Xu, G. Tao, Rethinking the Evaluation of Secure Code Generation, 2025. doi:10.48550/arXiv.2503.15554, arXiv:2503.15554 [cs].
- [6] Y. Shin, L. Williams, An empirical model to predict security vulnerabilities using code complexity metrics, in: *Proceedings of the Second ACM-IEEE international symposium on Empirical software engineering and measurement, ESEM '08*, Association for Computing Machinery, New York, NY, USA, 2008, pp. 315–317. URL: <https://dl.acm.org/doi/10.1145/1414004.1414065>. doi:10.1145/1414004.1414065.
- [7] V. J. Manès, H. Han, C. Han, S. K. Cha, M. Egele, E. J. Schwartz, M. Woo, The Art, Science, and Engineering of Fuzzing: A Survey, *IEEE Transactions on Software Engineering* 47 (2021) 2312–2331. URL: <https://ieeexplore.ieee.org/abstract/document/8863940>. doi:10.1109/TSE.2019.2946563.
- [8] M. Pistoia, S. Chandra, S. J. Fink, E. Yahav, A survey of static analysis methods for identifying security vulnerabilities in software systems, *IBM systems journal* 46 (2007) 265–288.
- [9] H. Choi, D. Kang, J.-Y. Choi, Static Analysis for Software Reliability and Security, in: K. Daimi, H. R. Arabnia, L. Deligiannidis, M.-S. Hwang, F. G. Tinetti (Eds.), *Advances in Security, Networks, and Internet of Things*, Springer International Publishing, Cham, 2021, pp. 463–470.
- [10] A. Gosain, G. Sharma, Static analysis: A survey of techniques and tools, in: *Intelligent Computing and Applications: Proceedings of the International Conference on ICA, 22-24 December 2014*, Springer, 2015, pp. 581–591.
- [11] Y. Shin, L. Williams, An empirical model to predict security vulnerabilities using code complexity metrics, in: *Proceedings of the Second ACM-IEEE international symposium on Empirical software engineering and measurement*, 2008, pp. 315–317.
- [12] S. Moshtari, A. Sami, M. Azimi, Using complexity metrics to improve software security, *Computer Fraud & Security* 2013 (2013) 8–17.
- [13] M. J. Tehrani, S. Hashemi, Assessing vulnerability in smart contracts: The role of code complexity metrics in security analysis, *arXiv preprint arXiv:2411.17343* (2024).
- [14] W. Charoenwet, P. Thongtanunam, V.-T. Pham, C. Treude, Toward effective secure code reviews: an empirical study of security-related coding weaknesses, *Empirical Software Engineering* 29 (2024) 88. URL: <https://doi.org/10.1007/s10664-024-10496-y>. doi:10.1007/s10664-024-10496-y.
- [15] A. Edmundson, B. Holtkamp, E. Rivera, M. Finifter, A. Mettler, D. Wagner, An Empirical Study on the Effectiveness of Security Code Review, in: J. Jürjens, B. Livshits, R. Scandariato (Eds.),

Engineering Secure Software and Systems, Springer, Berlin, Heidelberg, 2013, pp. 197–212. doi:10.1007/978-3-642-36563-8_14.

- [16] Y. Acar, M. Backes, S. Fahl, S. Garfinkel, D. Kim, M. L. Mazurek, C. Stransky, Comparing the Usability of Cryptographic APIs, in: 2017 IEEE Symposium on Security and Privacy (SP), 2017, pp. 154–171. URL: <https://ieeexplore.ieee.org/abstract/document/7958576>. doi:10.1109/SP.2017.52, ISSN: 2375-1207.
- [17] N. Perry, M. Srivastava, D. Kumar, D. Boneh, Do users write more insecure code with ai assistants?, in: Proceedings of the 2023 ACM SIGSAC conference on computer and communications security, 2023, pp. 2785–2799.
- [18] J. Becker, N. Rush, E. Barnes, D. Rein, Measuring the impact of early-2025 AI on experienced open-source developer productivity, arXiv preprint arXiv:2507.09089 (2025).
- [19] R. Balebako, A. Marsh, J. Lin, J. Hong, L. Faith Cranor, The Privacy and Security Behaviors of Smartphone App Developers, in: Proceedings 2014 Workshop on Usable Security, Internet Society, San Diego, CA, 2014. doi:10.14722/usec.2014.23006.
- [20] L. Braz, A. Bacchelli, Software security during modern code review: the developer’s perspective, in: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Association for Computing Machinery, New York, NY, USA, 2022, pp. 810–821. URL: <https://dl.acm.org/doi/10.1145/3540250.3549135>. doi:10.1145/3540250.3549135.
- [21] M. Gutfleisch, J. H. Klemmer, N. Busch, Y. Acar, M. A. Sasse, S. Fahl, How Does Usable Security (Not) End Up in Software Products? Results From a Qualitative Interview Study, in: 2022 IEEE Symposium on Security and Privacy (SP), 2022, pp. 893–910. URL: <https://ieeexplore.ieee.org/document/9833756>. doi:10.1109/SP46214.2022.9833756, ISSN: 2375-1207.
- [22] H. Assal, S. Chiasson, ’Think secure from the beginning’: A Survey with Software Developers, in: Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems, CHI ’19, Association for Computing Machinery, New York, NY, USA, 2019, pp. 1–13. URL: <https://dl.acm.org/doi/10.1145/3290605.3300519>. doi:10.1145/3290605.3300519.
- [23] M. Meli, M. R. McNiece, B. Reaves, How Bad Can It Get? Characterizing Secret Leakage in Public GitHub Repositories, in: Proceedings 2019 Network and Distributed System Security Symposium, Internet Society, San Diego, CA, 2019.
- [24] F. Fischer, K. Böttinger, H. Xiao, C. Stransky, Y. Acar, M. Backes, S. Fahl, Stack Overflow Considered Harmful? The Impact of Copy&Paste on Android Application Security, in: 2017 IEEE Symposium on Security and Privacy (SP), 2017, pp. 121–136. URL: <https://ieeexplore.ieee.org/abstract/document/7958574>. doi:10.1109/SP.2017.31, ISSN: 2375-1207.
- [25] C. Stransky, Y. Acar, D. C. Nguyen, D. Wermke, D. Kim, E. M. Redmiles, M. Backes, S. Garfinkel, M. L. Mazurek, S. Fahl, Lessons Learned from Using an Online Platform to Conduct Large-Scale, Online Controlled Security Experiments with Software Developers, 2017. URL: <https://www.usenix.org/conference/cset17/workshop-program/presentation/stransky>.
- [26] A. Alami, M. Zahedi, N. Ernst, Are You a Real Software Engineer? Best Practices in Online Recruitment for Software Engineering Studies, in: Proceedings of the 1st IEEE/ACM International Workshop on Methodological Issues with Empirical Studies in Software Engineering, WSESE ’24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 52–57. URL: <https://dl.acm.org/doi/10.1145/3643664.3648207>. doi:10.1145/3643664.3648207.
- [27] W. Wang, D. Hidellaarachchi, J. Grundy, H. Khalajzadeh, H. O. Obie, A. Madugalla, End-Users vs Software Practitioners: Recruitment Challenges and Strategies in Software Engineering Research, in: 2024 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC), 2024, pp. 400–411. URL: <https://ieeexplore.ieee.org/abstract/document/10714544>. doi:10.1109/VL/HCC60511.2024.00063, ISSN: 1943-6106.
- [28] H. Kaur, S. Klivan, D. Votipka, Y. Acar, S. Fahl, Where to Recruit for Security Development Studies: Comparing Six Software Developer Samples, 2022, pp. 4041–4058. URL: <https://www.usenix.org/conference/usenixsecurity22/presentation/kaur>.
- [29] M. Tahaei, K. Vaniea, Lessons Learned From Recruiting Participants With Programming Skills for

- Empirical Privacy and Security Studies: RoPES - ICSE 2022, 2022. URL: <https://ropes-workshops.github.io/ropes22/>.
- [30] R. Serafini, M. Gutfleisch, S. A. Horstmann, A. Naiakshina, On the Recruitment of Company Developers for Security Studies: Results from a Qualitative Interview Study, 2023, pp. 321–340. URL: <https://www.usenix.org/conference/soups2023/presentation/serafini>.
 - [31] G. Sandoval, H. Pearce, T. Nys, R. Karri, S. Garg, B. Dolan-Gavitt, Lost at C: A User Study on the Security Implications of Large Language Model Code Assistants, 2023, pp. 2205–2222. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/sandoval>.
 - [32] V. Belozarov, P. J. Barclay, A. Sami, Secure Coding with AI, From Creation to Inspection, arXiv preprint arXiv:2504.20814 (2025).
 - [33] G. Marvin, N. Hellen, D. Jjingo, J. Nakatumba-Nabende, Prompt Engineering in Large Language Models, in: I. J. Jacob, S. Piramuthu, P. Falkowski-Gilski (Eds.), Data Intelligence and Cognitive Informatics, Springer Nature, Singapore, 2024, pp. 387–402. doi:10.1007/978-981-99-7962-2_30.
 - [34] M. Bruni, F. Gabrielli, M. Ghafari, M. Kropp, Benchmarking Prompt Engineering Techniques for Secure Code Generation with GPT Models, in: 2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge), 2025, pp. 93–103. URL: <https://ieeexplore.ieee.org/abstract/document/11052790>. doi:10.1109/Forge66646.2025.00018.
 - [35] J. He, M. Vechev, Large Language Models for Code: Security Hardening and Adversarial Testing, in: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS '23, Association for Computing Machinery, New York, NY, USA, 2023, pp. 1865–1879. URL: <https://dl.acm.org/doi/10.1145/3576915.3623175>. doi:10.1145/3576915.3623175.
 - [36] J. He, M. Vero, G. Krasnopolska, M. Vechev, Instruction tuning for secure code generation, in: Proceedings of the 41st International Conference on Machine Learning, volume 235 of *ICML '24*, JMLR.org, Vienna, Austria, 2024, pp. 18043–18062.
 - [37] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, C. Xiong, CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis, 2023. URL: <http://arxiv.org/abs/2203.13474>. doi:10.48550/arXiv.2203.13474, arXiv:2203.13474 [cs].
 - [38] A. S. Molison, M. Moraes, G. Melo, F. Santos, W. K. G. Assuncao, Is LLM-Generated Code More Maintainable & Reliable than Human-Written Code?, 2025. doi:10.48550/arXiv.2508.00700, arXiv:2508.00700 [cs].
 - [39] D. Cotroneo, C. Improta, P. Liguori, Human-Written vs. AI-Generated Code: A Large-Scale Study of Defects, Vulnerabilities, and Complexity, in: 2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE), 2025, pp. 252–263. doi:10.1109/ISSRE66568.2025.00035, iSSN: 2332-6549.
 - [40] S. A. Licorish, A. Bajpai, C. Arora, F. Wang, K. Tantithamthavorn, Comparing Human and LLM Generated Code: The Jury is Still Out!, 2025. doi:10.48550/arXiv.2501.16857, arXiv:2501.16857 [cs].
 - [41] M. Astekin, M. Hort, L. Moonen, An Exploratory Study on How Non-Determinism in Large Language Models Affects Log Parsing, in: Proceedings of the ACM/IEEE 2nd International Workshop on Interpretability, Robustness, and Benchmarking in Neural Software Engineering, InteNSE '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 13–18. doi:10.1145/3643661.3643952.
 - [42] A. Y. Cui, P. Yu, Do Language Models Have Bayesian Brains? Distinguishing Stochastic and Deterministic Decision Patterns within Large Language Models, 2025. doi:10.48550/arXiv.2506.10268, arXiv:2506.10268 [cs].
 - [43] S. Sawadogo, A. Sabane, R. Kafando, A. K. Kabore, T. F. Bissyande, Revisiting the Non-Determinism of Code Generation by the GPT-3.5 Large Language Model, in: 2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), 2025, pp. 36–44. doi:10.1109/SANER64311.2025.00012, iSSN: 2640-7574.
 - [44] A.-M. Orloff, C. Tiefenau, M. Smith, SoK: I Have the (Developer) Power! Sample Size Estimation for Fisher's Exact, Chi-Squared, McNemar's, Wilcoxon Rank-Sum, Wilcoxon Signed-Rank and

- t-tests in Developer-Centered Usable Security, 2023, pp. 341–359. URL: <https://www.usenix.org/conference/soups2023/presentation/ortloff>.
- [45] S. Beckers, Large language models as nondeterministic causal models, arXiv preprint arXiv:2509.22297 (2025).
 - [46] Y. Song, G. Wang, S. Li, B. Y. Lin, The good, the bad, and the greedy: Evaluation of llms should not ignore non-determinism, in: Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), 2025, pp. 4195–4206.
 - [47] S. Ouyang, J. M. Zhang, M. Harman, M. Wang, An empirical study of the non-determinism of chatgpt in code generation, ACM Transactions on Software Engineering and Methodology 34 (2025) 1–28.
 - [48] Egli, Jasmine, LLMEvaluationTool/LLMgenerateCodeEvaluationTool at main · jaegli/LLMEvaluationTool, 2026. URL: <https://github.com/jaegli/LLMEvaluationTool/>.
 - [49] S. Gonzalez, Ruff: a game changer or Python linters., 2024. URL: <https://xantycg.medium.com/ruff-a-game-changer-for-python-linters-12b1ec8c5f12>.
 - [50] D. Walsh, Podman in Action: Secure, rootless containers for Kubernetes, microservices, and more, Simon and Schuster, 2023.
 - [51] AoC, Advent of Code 2024, 2025. URL: <https://adventofcode.com/2024/>.
 - [52] J. Börstler, K. E. Bennin, S. Hooshangi, J. Jeuring, H. Keuning, C. Kleiner, B. MacKellar, R. Duran, H. Störrle, D. Toll, Developers talking about code quality, Empirical Software Engineering 28 (2023) 128.
 - [53] OpenAI, Models, 2025. URL: <https://platform.openai.com/docs/models>.
 - [54] M. A. Alvarado Gonzalez, M. B. Hernandez, M. A. Peñaloza Perez, B. Lopez Orozco, J. Tadeo Cruz Soto, S. Malagon, Do Repetitions Matter? Strengthening Reliability in LLM Evaluations, arXiv e-prints (2025) arXiv: 2509.24086.
 - [55] T. Kohno, Y. Acar, W. Loh, Ethical Frameworks and Computer Security Trolley Problems: Foundations for Conversations, in: Usenix Security, 2023, pp. 5145–5162. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/kohno>.
 - [56] E. Verdier, The Fastest Way to Boost your Code Quality: Use Ruff Linter, 2023. URL: <https://data-ai.theodo.com/en/technical-blog/boost-code-quality-ruff-linter>.
 - [57] T. BRTNÍK, Analysis and Extension of the Ruff Linter (????).
 - [58] B. T. Kristinsson, Implementing Python code quality checks in the CMSSW Continuous Integration infrastructure, Technical Report, 2024.
 - [59] OpenAI, Codex, 2025. URL: <https://openai.com/de-DE/codex/>.
 - [60] A. Akamine, D. Hayashi, A. Tomizawa, Y. Nagasaki, C. Akamine, T. Fukawa, I. Hirose, O. Saigo, M. Hayashi, M. Nanaoya, Effects of temperature settings on information quality of ChatGPT-3.5 responses: A prospective, single-blind, observational cohort study, medRxiv (2024) 2024.06.11.24308759.
 - [61] OpenAI, createTranslation, ?????. URL: https://platform.openai.com/docs/api-reference/audio/createTranslation#audio_createTranslation-temperature.
 - [62] OpenAI, Create chat completion, ?????. URL: https://platform.openai.com/docs/api-reference/chat/create#chat_create-frequency_penalty.
 - [63] w3resource, w3resource, 2024. URL: <https://www.w3resource.com/python-exercises/cybersecurity/>.
 - [64] AoC, Advent of Code 2025, 2025. URL: <https://adventofcode.com/2025/>.

A. Appendix: Framework and Metrics

Execution and static analysis are coordinated by the `analyse_outputs` routine, which launches each script in its own container, executes it and performs static checks. With a minimal monitoring system, the `analyse_outputs` orchestration routine verifies required configurations (e.g., `ruff_cfg`, `output_directory`),

executes all Python scripts in the output directory, logs execution details and records per-run meta-data (started_at, exit_code, standard output (stdout), standard error (stderr), and duration_ms) in the podman_runs table.

Table 4

Error Type Distribution Across All Models and Prompts in Feasibility Study

Error Type	Count	Percentage (%)
No output	426	35.4
WrongLogic	394	32.7
SyntaxError	194	16.1
EOFError	70	5.8
NameError	49	4.1
OSError	19	1.6
IndexError	18	1.5
RecursionError	12	1.0
UnboundLocalError	11	0.9
ValueError	6	0.5
RuntimeError	4	0.3
TypeError	1	0.1
Total	1204	100.0

A.1. Ruff Linter Security Codes

For automated code-quality assessment, a linter in combination with custom functions were used. Linters detect syntax errors, security anti-patterns, or style violations[56]. After evaluating Pylint, Pyflakes, Pycodestyle, Flake8, Bandit, and Ruff, Ruff, a Rust-based linter, was selected for its execution speed and extensive rule set (800 rules, including Bandit security checks). [56, 57, 49, 58] Ruff was configured via pyproject.toml, mounted into rootless Podman containers, and executed through the lint_script_in_container function. Ruff findings and the custom metrics were recorded in the ruff_issues table; The analyse_outputs routine orchestrates logging, database connections, Podman volume mounts, script execution, and linting (including custom metric calculation).

The Ruff Security Codes discussed in this paper are as follows:

- S101 (assert used): Use of the assert statement. While common in testing, it is flagged in production code because assert statements can be optimized away when Python is run with the optimize flag, potentially bypassing critical security checks.
- S105 (hardcoded password string): Possible hardcoded password or credential. This is triggered when the linter finds a string assignment to a variable name that looks like a secret (e.g., password = "12345").
- S113 (request without timeout): Probable use of a requests call without a timeout parameter. This prevents your application from hanging indefinitely if a server fails to respond.
- S311 (suspicious non cryptographic random usage): Use of the standard random library for security or cryptographic purposes. The random module is pseudo-random and predictable; for security (like tokens or passwords), secrets or os.urandom should be used instead.
- S324 (hashlib insecure hash function): Use of insecure hash functions like MD5 or SHA1 for sensitive data. These functions are vulnerable to collision attacks and should be replaced with stronger algorithms.

B. Appendix: LLM Information

Five LLM model variants were compared: gpt-4.1, gpt-4o-mini, gpt-5.1, gpt-5-mini, and gpt-5-nano. According to the OpenAI documentation[59]. These models differ in coding reasoning, determinism,

latency, cost per run, error profile or failure modes. The following claims are made about the models: gpt-5.1 and gpt-5-mini offer the highest coding and reasoning quality (but at higher cost and latency), while gpt-5-nano and gpt-4o-mini prioritize very low cost and fast throughput with reduced reasoning depth. gpt-4.1 provides a stable, more deterministic baseline useful for reproducible comparisons between stochastic, higher-capability and low-cost models.[53].

model	gpt-4.1	gpt-4o-mini	gpt-5-mini	gpt-5-nano	gpt-5.1
date	2025-04-14	2024-07-18	2025-08-07	2025-08-07	2025-11-13

Table 5
Model names and dates

Execution and static analysis are coordinated by the `analyse_outputs` routine, which launches each script in its own container, executes it and performs static checks. With a minimal monitoring system, the `analyse_outputs` orchestration routine verifies required configurations (e.g., `ruff_cfg`, `output_directory`), executes all Python scripts in the output directory, logs execution details and records per-run metadata —`started_at`, `exit_code`, standard output (`stdout`), standard error (`stderr`), and `duration_ms` —in the `podman_runs` table.

Request parameter	Set value	Description
client	OpenAI	Official OpenAI HTTP client used to send requests and receive responses and sets uniform client configuration.
model	e.g. gpt-4.1	LLM variants used for a given generation.
temperature	0.0	To minimize randomness. A value of 0 supports determinism. [60]
top_p	1.0	Nucleus sampling cutoff; 1.0 disables top-p filtering so all tokens are considered.[61]
max_tokens	15000	Max. number of response tokens.
frequency_penalty	0.0	Penalizes repeated tokens; set to 0 to avoid altering model repetition behavior.[62]
presence_penalty	0.0	Penalizes introducing new topics; set to 0 to avoid discouraging content variation.[62]

Table 6
To achieve maximal determinism,^{[24][25][26]} the input variables were controlled and the generations were issued via the OpenAI API with fixed request parameters.

Response parameter	Value	Description
stop_reason	stop,length, timeout, error.	Reason the API reports why generation ended: ‘stop’ indicates normal completion.
request_id	uid	Unique identifier for the API request (useful for tracing and support).
response_id	uid	Unique identifier for the specific response object returned by the API (links request to output).
created_at	timestamp	Timestamp when the response was created by the API server
Latency_ms	in milliseconds	Custom: Round-trip time in milliseconds between sending the request and receiving the response; used to monitor performance and variability.

Table 7
In order to achieve maximal determinism, the output variables from the generation via OpenAI API were defined as response parameters.

C. Appendix: Coding Exercises

The coding challenges are taken from the following resources: w3resource[63], Advent of Code 2024[51], and Advent of Code 2025[64]. They are described in Table 8.

prompt_id	Exercise name	Complexity level	Link to original exercise & code
2	AoC 2024 Day 17	Advanced	GitHub: FelixCenusa
3	AoC 2024 Day 21	Advanced	GitHub: FelixCenusa
4	Leaked Password Checker	Intermediate: Involves external API integration, including handling HTTP requests and responses. Requires understanding data formats (e.g., JSON) and dealing with potential edge cases, making this exercise significantly more complex.	w3resource Ex 6
5	SHA-256 Hasher	Intermediate: Requires understanding hashing concepts, managing input/output in Python, and utilizing the hashlib library. Implementation is straightforward but requires attention to detail.	w3resource Ex 1
6	Random Password Generators	Basic: Involves random number generation and string manipulation.	w3resource Ex 2
7	Common Substitution Generator	Intermediate: Requires an understanding of character mapping and the implications of substitutions in password strength.	w3resource Ex 4
8	Password File Generator	Involves file I/O and validation logic.	w3resource Ex 5
9	Password strength meter	Basic	w3resource Ex 7
10	Brute Force Attack	Intermediate	w3resource Ex 10
11	AoC 2024 Day 1	Advanced: requires understanding both sorting list and iterating through lists, along with basic arithmetic.	GitHub: Solution11a, GitHub: Solution11b, GitHub: Solution11c, GitHub: Solution11d
12	AoC 2024 Day 8	Advanced: Understanding of programming skills (mathematical calculations, nested loops, conditions checks, edge cases as well as geometric concepts like antinodes).	GitHub: Solution12a, GitHub: Solution12b, GitHub: Solution12c
13	AoC 2025 Day 10	Advanced: implementing graph traversal algorithm or similar methods.	GitHub: Solution13a, GitHub: Solution13d, GitHub: Solution13e
14	Password Criteria Checker	Basic: Involves conditional statements and string validation techniques. Requires systematic logic to check multiple criteria, making it slightly more challenging than basic syntax.	w3resource Ex 3
16	AoC 2025 Day 1	Advanced	No reference
17	AoC 2025 Day 2	Advanced	No reference

Table 8

Summary of Cybersecurity and Advent of Code (AoC) programming exercises, categorized by complexity and source.